PRAKTIKUM 4
SINGLY LINKED LIST

NAMA: Ihsanta Zaki Sanjaya

KELAS: 1 D4 Teknik Komputer B

NRP: 3223600058

DOSEN: Dr Bima Sena Bayu Dewantara S.ST, M.T

MATA KULIAH: Praktikum Pemograman Dasar 2

TANGGAL: Rabu, 27 Maret 2024

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2024

Praktikum 4
Singly Linked List

Untuk memberikan pemahaman kepada anda tentang konsep dasar Singly Linked List dan cara menggunakan dalam program C++, maka cobalah beberapa modul percobaan dibawah ini.

1. **Percobaan 1 – Inisialisasi Singly Linked List**
   Pada percobaan ini, sebuah Singly Linked List (SLL) dideklarasikan dan diinisialisasi dengan menggunakan 2 jenis pemrograman, yaitu C dan C++.

   a. **Menggunakan C**

```
// A simple C program to introduce
// a linked list
#include <stdio.h>
#include <stdlib.h>

struct Node {
	int data;
	struct Node* next;
};

// Program to create a simple linked
// list with 3 nodes
int main()
{
	struct Node* head = NULL;
	struct Node* second = NULL;
	struct Node* third = NULL;

	// allocate 3 nodes in the heap
	head = (struct Node*)malloc(sizeof(struct Node));
	second = (struct Node*)malloc(sizeof(struct Node));
	third = (struct Node*)malloc(sizeof(struct Node));

	/* Three blocks have been allocated dynamically.
	We have pointers to these three blocks as head,
	second and third
	head		second		third
```

```
                |                       |                       |
                |                       |                       |
        +---+---- +       +----+----+       +----+--- +
        | # | # |         | # | # |         | # | # |
        +---+---- +       +----+----+       +----+--- +

# represents any random value.
Data is random because we haven't assigned
anything yet */

        head->data = 1; // assign data in first node
        head->next = second; // Link first node with
        // the second node

        /* data has been assigned to the data part of the first
        block (block pointed by the head). And next
        pointer of first block points to second.
        So they both are linked.

        head            second                  third
                |                       |                       |
                |                       |                       |
        +---+---+         +----+----+       +-----+--- +
        | 1 |   o ---->   | # | # |         | # | # |
        +---+---+         +----+----+       +-----+--- +
*/

        // assign data to second node
        second->data = 2;

        // Link second node with the third node
        second->next = third;

        /* data has been assigned to the data part of the second
        block (block pointed by second). And next
        pointer of the second block points to the third
        block. So all three blocks are linked.

        head            second                  third
                |                       |                       |
                |                       |                       |
```

```
        +---+---+          +---+---+          +----+--- +
        | 1 |   o ---->    | 2 |   o---- >    | # | # |
        +---+-- +          +---+ --+          +----+----+        */

        third->data = 3; // assign data to third node
        third->next = NULL;

        /* data has been assigned to data part of third
        block (block pointed by third). And next pointer
        of the third block is made NULL to indicate
        that the linked list is terminated here.

        We have the linked list ready.

                head
                        |
                        |
                +---+---+          +---+---+          +----+ -----+
                | 1 |   o---- >    | 2 |   o ---->    | 3 | NULL |
                +---+---+          +---+---+          +----+ -----+


        Note that only head is sufficient to represent
        the whole list. We can traverse the complete
        list by following next pointers. */

        return 0;
}
```

Hasil:

Praktikum > C P4 1.1.c > main()

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
int data;
struct Node* next;
};
int main ()

{
struct Node* head = NULL;
struct Node* second = NULL;
struct Node* third = NULL;

head = (struct Node*)malloc(sizeof(struct Node));
second = (struct Node*)malloc(sizeof(struct Node));
third = (struct Node*)malloc(sizeof(struct Node));

head->data = 1;
head->next = second;

second->data = 2;
second->next = third;

third->data = 3;
third->next = NULL;

return 0;
```

PROBLEMS OUTPUT DEBUG CONSOLE PORTS

Filter (e.g. text, !exclude)

```
Type "apropos word" to search for commands related to "word".
Warning: Debuggee TargetArchitecture not detected, assuming x86_64.
=cmd-param-changed,param="pagination",value="off"
[New Thread 9848.0x38a8]

Thread 1 hit Breakpoint 1, main () at D:\1. KULIAH PENS\Semester II\Progdas 2\Praktikum\P4 1.1.c:11
11      struct Node* head = NULL;
Loaded 'C:\WINDOWS\SYSTEM32\ntdll.dll'. Symbols loaded.
Loaded 'C:\WINDOWS\System32\kernel32.dll'. Symbols loaded.
Loaded 'C:\WINDOWS\System32\KernelBase.dll'. Symbols loaded.
Loaded 'C:\WINDOWS\System32\ucrtbase.dll'. Symbols loaded.
[Thread 9848.0x38a8 exited with code 0]
[Inferior 1 (process 9848) exited normally]
The program 'D:\1. KULIAH PENS\Semester II\Progdas 2\Praktikum\P4 1.1.exe' has exited with code 0 (0x00000000).
```

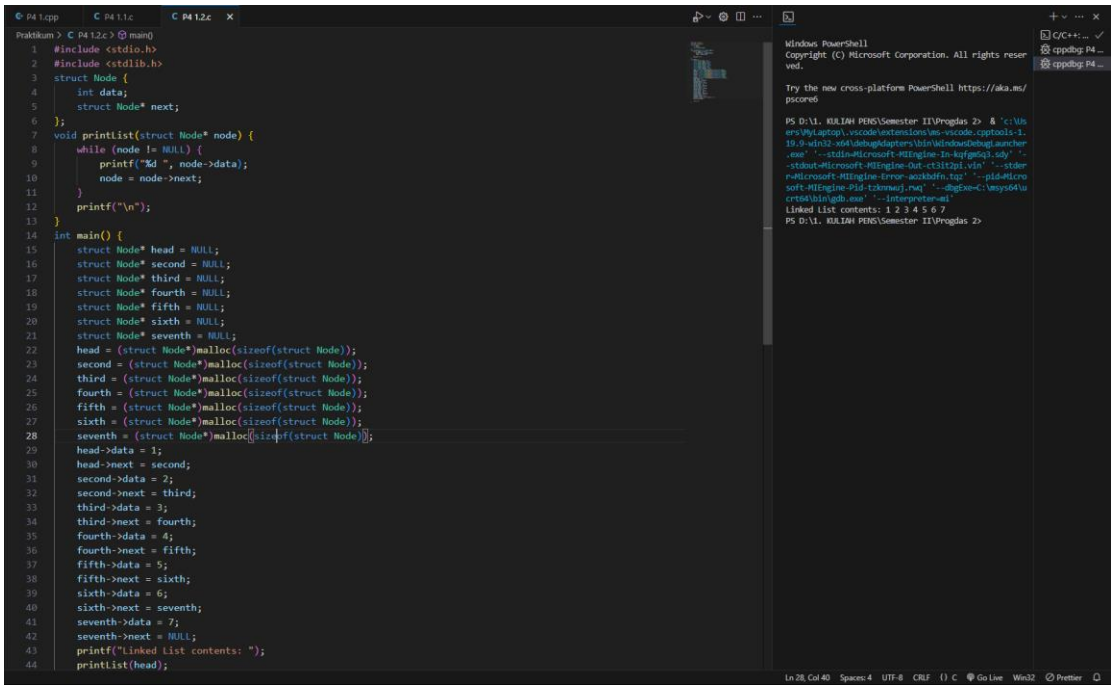
Dari percobaan di atas, coba tambahkan beberapa node baru misalnya fourth, fifth, sixth, dan seventh! Masing-masing isilah dengan data 4, 5, 6 dan 7.

Kode pemograman dengan tambahan data 4,5,6,7:

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* next;
};
void printList(struct Node* node) {
    while (node != NULL) {
        printf("%d ", node->data);
        node = node->next;
    }
    printf("\n");
}
int main() {
    struct Node* head = NULL;
    struct Node* second = NULL;
    struct Node* third = NULL;
```

```
    struct Node* fourth = NULL;
    struct Node* fifth = NULL;
    struct Node* sixth = NULL;
    struct Node* seventh = NULL;
    head = (struct Node*)malloc(sizeof(struct Node));
    second = (struct Node*)malloc(sizeof(struct Node));
    third = (struct Node*)malloc(sizeof(struct Node));
    fourth = (struct Node*)malloc(sizeof(struct Node));
    fifth = (struct Node*)malloc(sizeof(struct Node));
    sixth = (struct Node*)malloc(sizeof(struct Node));
    seventh = (struct Node*)malloc(sizeof(struct Node));
    head->data = 1;
    head->next = second;
    second->data = 2;
    second->next = third;
    third->data = 3;
    third->next = fourth;
    fourth->data = 4;
    fourth->next = fifth;
    fifth->data = 5;
    fifth->next = sixth;
    sixth->data = 6;
    sixth->next = seventh;
    seventh->data = 7;
    seventh->next = NULL;
    printf("Linked List contents: ");
    printList(head);

    return 0;
}
```

Hasil Pemograman:



b. Menggunakan C++

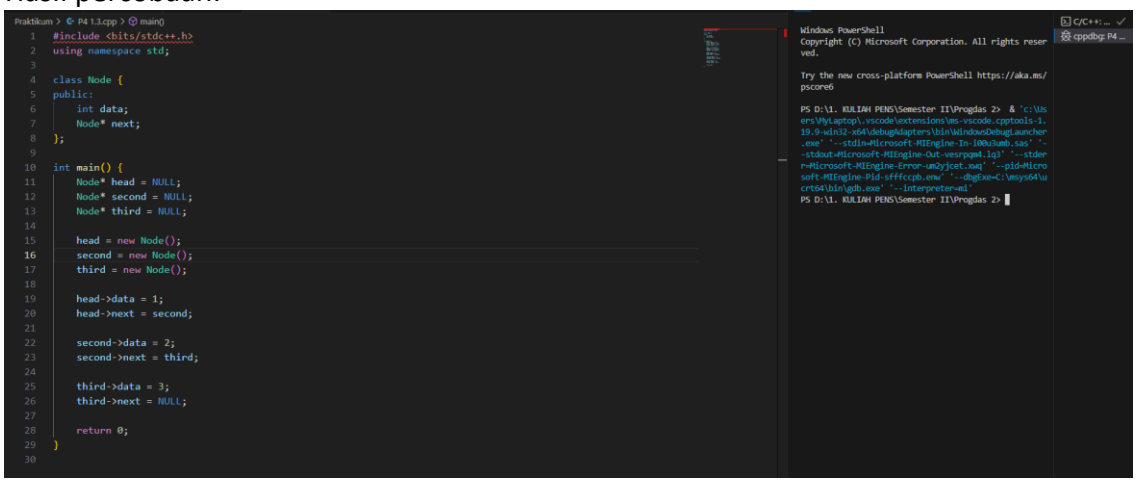
```
// A simple CPP program to introduce
// a linked list
#include <bits/stdc++.h>
using namespace std;

class Node {
public:
        int data;
```

```
        Node* next;
};

// Program to create a simple linked
// list with 3 nodes
int main()
{
        Node* head = NULL;
        Node* second = NULL;
        Node* third = NULL;

        // allocate 3 nodes in the heap
        head = new Node();
        second = new Node();
        third = new Node();

        /* Three blocks have been allocated dynamically.
        We have pointers to these three blocks as head,
        second and third
        head            second                  third
                |                       |                       |
                |                       |                       |
        +---+---- +     +----+ ---+     +----+--- +
        | # | # |       | # | # |       | # | # |
        +---+---- +     +----+ ---+     +----+--- +

# represents any random value.
Data is random because we haven't assigned
anything yet */

        head->data = 1; // assign data in first node
        head->next = second; // Link first node with
        // the second node

        /* data has been assigned to the data part of first
        block (block pointed by the head). And next
        pointer of the first block points to second.
        So they both are linked.

        head            second                  third
                |                       |                       |
```

```
                |                       |                       |
        +---+---+         +----+ ---+       +-----+--- +
        | 1 |   o ---->   | # | # |         | # | # |
        +---+---+         +----+ ---+       +-----+--- +
*/

        // assign data to second node
        second->data = 2;

        // Link second node with the third node
        second->next = third;

        /* data has been assigned to the data part of the second
        block (block pointed by second). And next
        pointer of the second block points to the third
        block. So all three blocks are linked.

        head            second                  third
                |                       |                       |
                |                       |                       |
        +---+---+         +---+---+         +----+--- +
        | 1 |   o ---->   | 2 |   o---- >   | # | # |
        +---+---+         +---+---+         +----+--- +       */

        third->data = 3; // assign data to third node
        third->next = NULL;

        /* data has been assigned to the data part of the third
        block (block pointed by third). And next pointer
        of the third block is made NULL to indicate
        that the linked list is terminated here.

        We have the linked list ready.

                head
                        |
                        |
                +---+---+         +---+---+         +----+ -----+
                | 1 |   o---- >   | 2 |   o ---->   | 3 | NULL |
                +---+---+         +---+---+         +----+ -----+
```

```
        Note that only the head is sufficient to represent
        the whole list. We can traverse the complete
        list by following the next pointers. */

        return 0;
}
```

Hasil percobaan:



Dari percobaan di atas, coba tambahkan beberapa node baru misalnya fourth, fifth, sixth, dan seventh! Masing-masing isilah dengan data 4, 5, 6 dan 7.

Kode pemograman:

```
#include <bits/stdc++.h>
using namespace std;
class Node {
public:
    int data;
    Node* next;
};
void printList(Node* node) {
    while (node != NULL) {
        cout << node->data << " ";
        node = node->next;
    }
    cout << endl;
}
int main() {
    Node* head = NULL;
    Node* second = NULL;
    Node* third = NULL;
    Node* fourth = NULL;
    Node* fifth = NULL;
    Node* sixth = NULL;
```

```
    Node* seventh = NULL;
    head = new Node();
    second = new Node();
    third = new Node();
    fourth = new Node();
    fifth = new Node();
    sixth = new Node();
    seventh = new Node();

    head->data = 1;
    head->next = second;
    second->data = 2;
    second->next = third;
    third->data = 3;
    third->next = fourth;
    fourth->data = 4;
    fourth->next = fifth;
    fifth->data = 5;
    fifth->next = sixth;
    sixth->data = 6;
    sixth->next = seventh;
    seventh->data = 7;
    seventh->next = NULL;
    cout << "Linked List contents: ";
    printList(head);
    return 0;
}
```

Hasil percobaan:

Praktikum > P4 1.4.cpp > printList(Node *)

```
#include <bits/stdc++.h>
using namespace std;
class Node {
public:
    int data;
    Node* next;
};
void printList(Node* node) {
    while (node != NULL) {
        cout << node->data << " ";
        node = node->next;
    }
    cout << endl;
}
int main() {
    Node* head = NULL;
    Node* second = NULL;
    Node* third = NULL;
    Node* fourth = NULL;
    Node* fifth = NULL;
    Node* sixth = NULL;
    Node* seventh = NULL;
    head = new Node();
    second = new Node();
    third = new Node();
    fourth = new Node();
    fifth = new Node();
    sixth = new Node();
    seventh = new Node();

    head->data = 1;
    head->next = second;
    second->data = 2;
    second->next = third;
    third->data = 3;
    third->next = fourth;
    fourth->data = 4;
    fourth->next = fifth;
    fifth->data = 5;
    fifth->next = sixth;
    sixth->data = 6;
    sixth->next = seventh;
    seventh->data = 7;
    seventh->next = NULL;
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reser
ved.

Try the new cross-platform PowerShell https://aka.ms/
pscore6

PS D:\1. KULIAH PENS\Semester II\Progdas 2>  & 'c:\Us
ers\MyLaptop\.vscode\extensions\ms-vscode.cpptools-1.
19.9-win32-x64\debugAdapters\bin\WindowsDebugLauncher
.exe' '--stdin=Microsoft-MIEngine-In-pjea4j0p.zuv' '-
-stdout=Microsoft-MIEngine-Out-h4qnbdqr.gsk' '--stder
r=Microsoft-MIEngine-Error-vogsnzts.kon' '--pid=Micro
soft-MIEngine-Pid-inpuho0y.0od' '--dbgExe=C:\msys64\u
crt64\bin\gdb.exe' '--interpreter=mi'
Linked List contents: 1 2 3 4 5 6 7
PS D:\1. KULIAH PENS\Semester II\Progdas 2>
```

Ln 12, Col 6 Spaces: 4 UTF-8 CRLF {} C++ Go Live Win32

c. **Tugas:** Sebutkan dan jelaskan 2 perbedaan dari pemrograman Singly Linked List menggunakan C dan C++!

C:
Pada pemrograman Singly Linked List menggunakan C, pengelolaan memori secara manual menggunakan fungsi-fungsi seperti malloc() dan free(). Programmer bertanggung jawab untuk mengalokasikan dan membebaskan memori secara eksplisit untuk setiap node.
C++:
Dalam C++, pengelolaan memori lebih mudah karena dapat menggunakan operator new untuk mengalokasikan memori secara dinamis dan operator delete untuk membebaskan memori. Dengan demikian, programmer tidak perlu secara eksplisit mengurus alokasi dan dealokasi memori seperti pada C. OOP (Object-Oriented Programming) Features: C: Pemrograman Singly Linked List menggunakan C tidak mendukung konsep OOP secara langsung. Struktur data dan fungsi-fungsi yang beroperasi pada linked list biasanya terpisah dan tidak terkait sebagai satu kesatuan yang terintegrasi. C++: Dalam C++, Singly Linked List dapat diimplementasikan sebagai kelas (class) dengan data anggota (member data) berupa pointer ke node pertama (head) dan fungsi-fungsi anggota (member functions) seperti insert, delete, search, dll. Hal ini memungkinkan untuk menggunakan fitur-fitur OOP seperti enkapsulasi, pewarisan, dan polimorfisme dalam pengelolaan linked list.

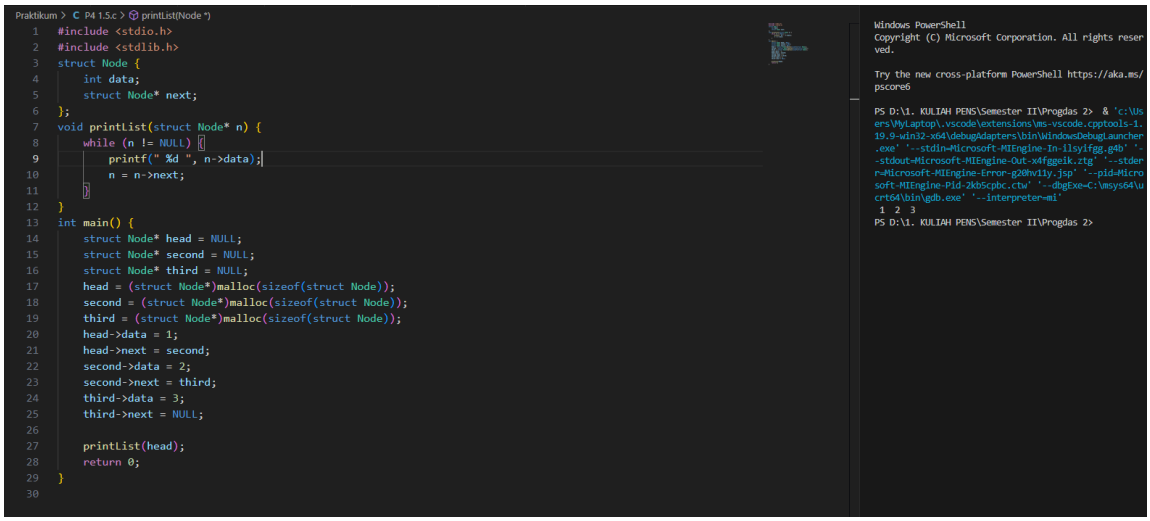
2. **Percobaan 2 – Melewatkan Linked List ke Fungsi Lain (untuk mencetak)**
Pada percobaan ini, dicoba untuk melewatkan node-node dalam Singly Linked List ke dalam fungsi untuk mencetak printList() dengan menggunakan 2 jenis pemrograman, yaitu C dan C++.

a. Menggunakan C

```
// A simple C program for traversal of a linked list
#include <stdio.h>
#include <stdlib.h>

struct Node {
	int data;
	struct Node* next;
};

// This function prints contents of linked list starting from
// the given node
void printList(struct Node* n)
{
	while (n != NULL) {
		printf(" %d ", n->data);
		n = n->next;
	}
}

int main()
{
```

```
        struct Node* head = NULL;
        struct Node* second = NULL;
        struct Node* third = NULL;

        // allocate 3 nodes in the heap
        head = (struct Node*)malloc(sizeof(struct Node));
        second = (struct Node*)malloc(sizeof(struct Node));
        third = (struct Node*)malloc(sizeof(struct Node));

        head->data = 1; // assign data in first node
        head->next = second; // Link first node with second

        second->data = 2; // assign data to second node
        second->next = third;

        third->data = 3; // assign data to third node
        third->next = NULL;

        printList(head);

        return 0;
}
```

Hasil percobaan:



Dari percobaan di atas, coba tambahkan beberapa node baru misalnya fourth, fifth, sixth, dan seventh! Masing-masing isilah dengan data 4, 5, 6 dan 7.

Kode percobaan:

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* next;
};
```

```c
void printList(struct Node* n) {
    while (n != NULL) {
        printf("%d ", n->data);
        n = n->next;
    }
}
int main() {
    struct Node* head = NULL;
    struct Node* second = NULL;
    struct Node* third = NULL;
    struct Node* fourth = NULL;
    struct Node* fifth = NULL;
    struct Node* sixth = NULL;
    struct Node* seventh = NULL;
    head = (struct Node*)malloc(sizeof(struct Node));
    second = (struct Node*)malloc(sizeof(struct Node));
    third = (struct Node*)malloc(sizeof(struct Node));
    fourth = (struct Node*)malloc(sizeof(struct Node));
    fifth = (struct Node*)malloc(sizeof(struct Node));
    sixth = (struct Node*)malloc(sizeof(struct Node));
    seventh = (struct Node*)malloc(sizeof(struct Node));
    head->data = 1;
    head->next = second;
    second->data = 2;
    second->next = third;
    third->data = 3;
    third->next = fourth;
    fourth->data = 4;
    fourth->next = fifth;
    fifth->data = 5;
    fifth->next = sixth;
    sixth->data = 6;
    sixth->next = seventh;
    seventh->data = 7;
    seventh->next = NULL;

    printList(head);
    return 0;
}
```

Hasil percobaan:

Praktikum > P4 1.5.c > main()

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* next;
};
void printList(struct Node* n) {
    while (n != NULL) {
        printf("%d ", n->data);
        n = n->next;
    }
}
int main() {
    struct Node* head = NULL;
    struct Node* second = NULL;
    struct Node* third = NULL;
    struct Node* fourth = NULL;
    struct Node* fifth = NULL;
    struct Node* sixth = NULL;
    struct Node* seventh = NULL;
    head = (struct Node*)malloc(sizeof(struct Node));
    second = (struct Node*)malloc(sizeof(struct Node));
    third = (struct Node*)malloc(sizeof(struct Node));
    fourth = (struct Node*)malloc(sizeof(struct Node));
    fifth = (struct Node*)malloc(sizeof(struct Node));
    sixth = (struct Node*)malloc(sizeof(struct Node));
    seventh = (struct Node*)malloc(sizeof(struct Node));
    head->data = 1;
    head->next = second;
    second->data = 2;
    second->next = third;
    third->data = 3;
    third->next = fourth;
    fourth->data = 4;
    fourth->next = fifth;
    fifth->data = 5;
    fifth->next = sixth;
    sixth->data = 6;
    sixth->next = seventh;
    seventh->data = 7;
    seventh->next = NULL;

    printList(head);
    return 0;
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reser
ved.

Try the new cross-platform PowerShell https://aka.ms/
pscore6

PS D:\1. KULIAH PENS\Semester II\Progdas 2>  & 'c:\Us
ers\MyLaptop\.vscode\extensions\ms-vscode.cpptools-1.
19.9-win32-x64\debugAdapters\bin\WindowsDebugLauncher
.exe' '--stdin=Microsoft-MIEngine-In-ilsyifgg.g4b' '-
-stdout=Microsoft-MIEngine-Out-x4fggeik.ztg' '--stder
r=Microsoft-MIEngine-Error-g20hvi1y.jsp' '--pid=Micro
soft-MIEngine-Pid-2kb5cpbc.ctw' '--dbgExe=C:\msys64\u
crt64\bin\gdb.exe' '--interpreter=mi'
 1  2  3
PS D:\1. KULIAH PENS\Semester II\Progdas 2>  & 'c:\Us
ers\MyLaptop\.vscode\extensions\ms-vscode.cpptools-1.
19.9-win32-x64\debugAdapters\bin\WindowsDebugLauncher
.exe' '--stdin=Microsoft-MIEngine-In-vw4dl42n.zb1' '-
-stdout=Microsoft-MIEngine-Out-zhefys3d.fux' '--stder
r=Microsoft-MIEngine-Error-0kecgxu3.uqj' '--pid=Micro
soft-MIEngine-Pid-dsh1xm34.a1f' '--dbgExe=C:\msys64\u
crt64\bin\gdb.exe' '--interpreter=mi'
1 2 3 4 5 6 7
PS D:\1. KULIAH PENS\Semester II\Progdas 2>
```

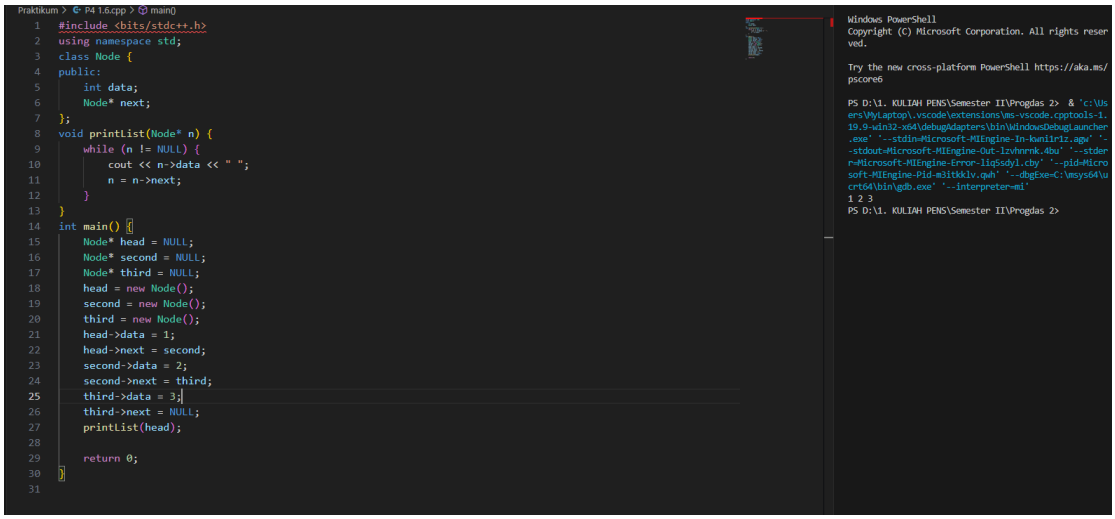
Ln 39, Col 27 Spaces: 4 UTF-8 CRLF {} C Go Live Win32

b. Menggunakan C++

```cpp
// A simple C++ program for traversal of a linked list
#include <bits/stdc++.h>
using namespace std;

class Node {
public:
 int data;
 Node* next;
};

// This function prints contents of linked list
// starting from the given node
void printList(Node* n)
{
```

```
while (n != NULL) {
        cout << n->data << " ";
        n = n->next;
}
}

// Driver code
int main()
{
Node* head = NULL;
Node* second = NULL;
Node* third = NULL;

// allocate 3 nodes in the heap
head = new Node();
second = new Node();
third = new Node();

head->data = 1; // assign data in first node
head->next = second; // Link first node with second

second->data = 2; // assign data to second node
second->next = third;

third->data = 3; // assign data to third node
third->next = NULL;

printList(head);

return 0;
}
```

Hasil percobaan:



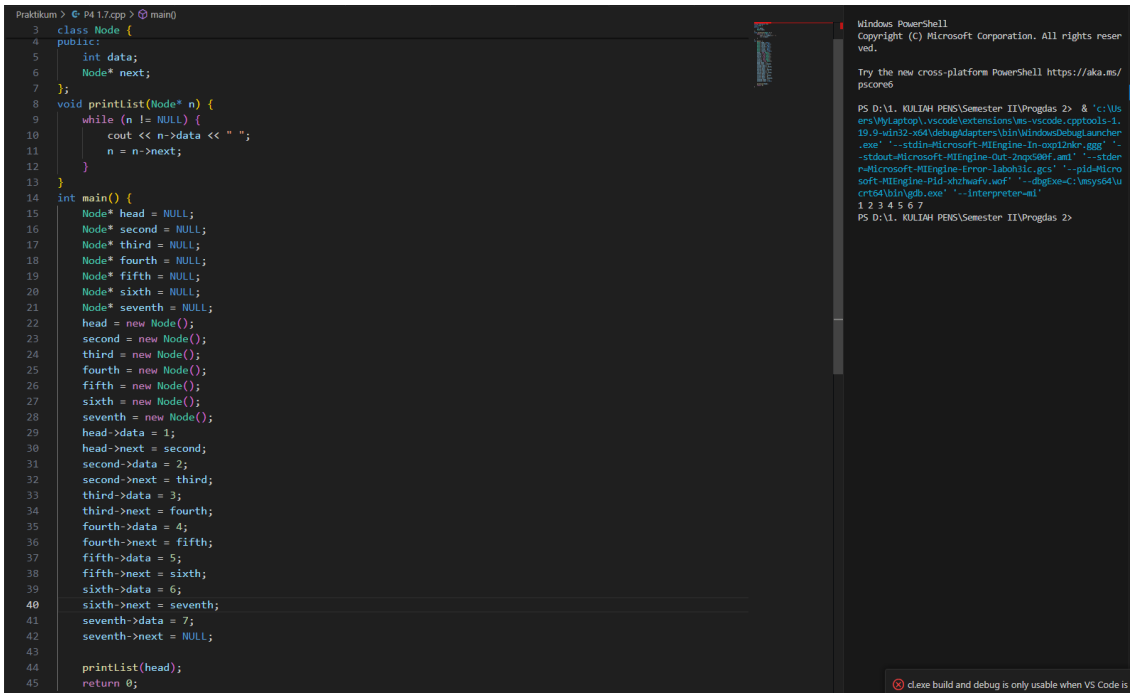
Dari percobaan di atas, coba tambahkan beberapa node baru misalnya fourth, fifth, sixth, dan seventh! Masing-masing isilah dengan data 4, 5, 6 dan 7.

Kode percobaan:

```
#include <bits/stdc++.h>
using namespace std;
class Node {
public:
    int data;
    Node* next;
};
void printList(Node* n) {
    while (n != NULL) {
        cout << n->data << " ";
        n = n->next;
    }
}
int main() {
    Node* head = NULL;
    Node* second = NULL;
    Node* third = NULL;
    Node* fourth = NULL;
    Node* fifth = NULL;
    Node* sixth = NULL;
    Node* seventh = NULL;
    head = new Node();
    second = new Node();
    third = new Node();
    fourth = new Node();
    fifth = new Node();
    sixth = new Node();
    seventh = new Node();
    head->data = 1;
    head->next = second;
    second->data = 2;
    second->next = third;
    third->data = 3;
    third->next = fourth;
    fourth->data = 4;
    fourth->next = fifth;
    fifth->data = 5;
    fifth->next = sixth;
    sixth->data = 6;
    sixth->next = seventh;
    seventh->data = 7;
```

```
    seventh->next = NULL;

    printList(head);
    return 0;
}
```

Hasil percobaan:



c. Tugas:

i. Tambahkan 1 baris perintah dalam fungsi printList() untuk mencetak alamat dari setiap node! Contoh hasil cetak:

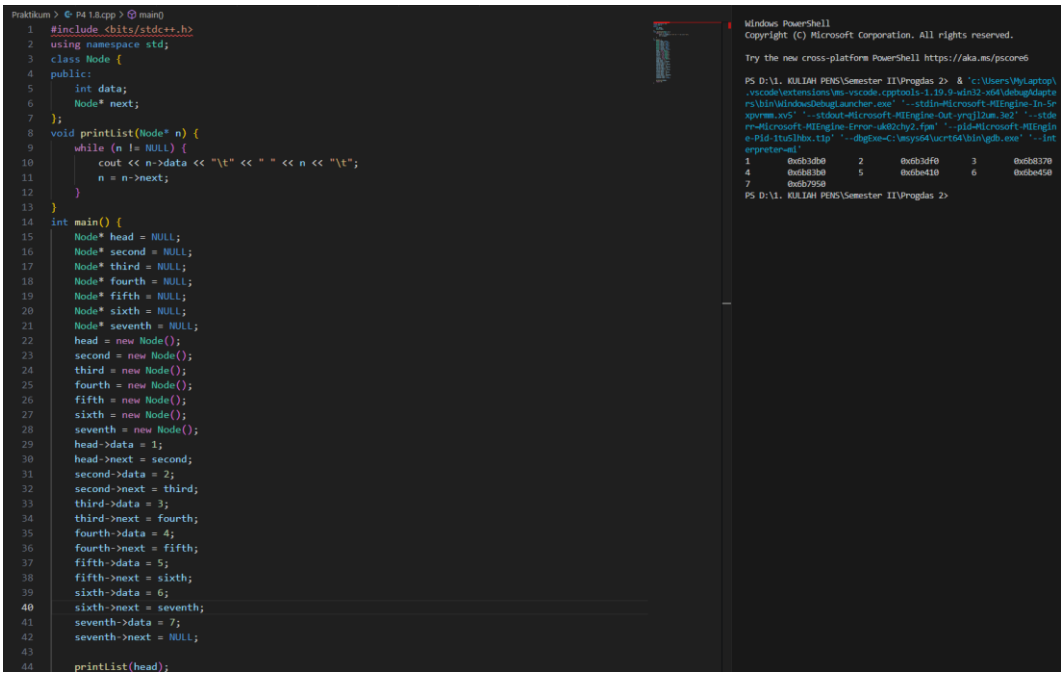
1	**15604112**	**2**	**15604128**	**3**	**15604144**

Kode pemograman:

```
#include <bits/stdc++.h>
using namespace std;
class Node {
public:
    int data;
    Node* next;
};
void printList(Node* n) {
    while (n != NULL) {
        cout << n->data << "\t" << " " << n << "\t";
        n = n->next;
    }
}
int main() {
    Node* head = NULL;
    Node* second = NULL;
```

```
    Node* third = NULL;
    Node* fourth = NULL;
    Node* fifth = NULL;
    Node* sixth = NULL;
    Node* seventh = NULL;
    head = new Node();
    second = new Node();
    third = new Node();
    fourth = new Node();
    fifth = new Node();
    sixth = new Node();
    seventh = new Node();
    head->data = 1;
    head->next = second;
    second->data = 2;
    second->next = third;
    third->data = 3;
    third->next = fourth;
    fourth->data = 4;
    fourth->next = fifth;
    fifth->data = 5;
    fifth->next = sixth;
    sixth->data = 6;
    sixth->next = seventh;
    seventh->data = 7;
    seventh->next = NULL;

    printList(head);
    return 0;
}
```

Hasil pemograman:



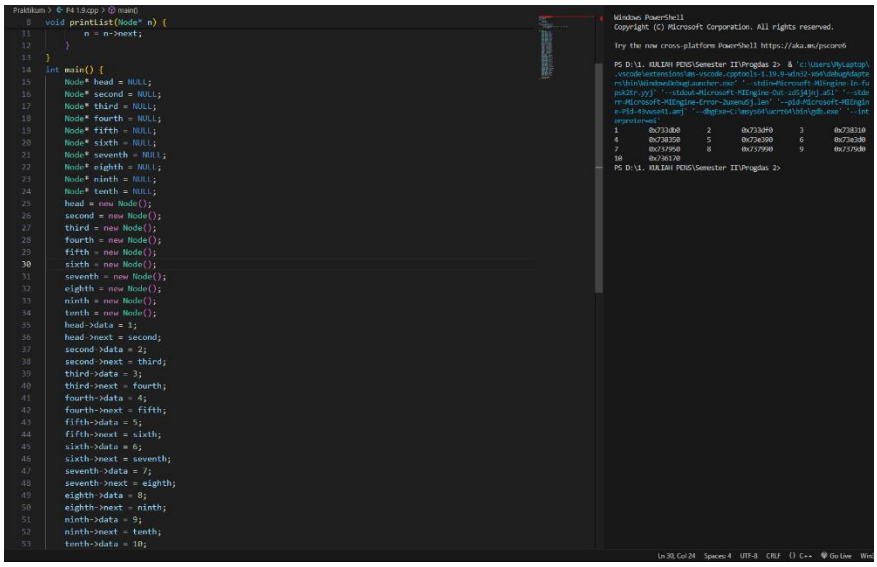
ii. Ulangi langkah (i) untuk penambahan 4 node baru seperti pada percobaan!

Kode pemograman:

```
#include <bits/stdc++.h>
using namespace std;
class Node {
public:
    int data;
    Node* next;
};
void printList(Node* n) {
    while (n != NULL) {
        cout << n->data << "\t" << " " << n << "\t";
        n = n->next;
    }
}
int main() {
    Node* head = NULL;
    Node* second = NULL;
    Node* third = NULL;
    Node* fourth = NULL;
    Node* fifth = NULL;
    Node* sixth = NULL;
    Node* seventh = NULL;
    Node* eighth = NULL;
    Node* ninth = NULL;
    Node* tenth = NULL;
    head = new Node();
    second = new Node();
    third = new Node();
```

```
    fourth = new Node();
    fifth = new Node();
    sixth = new Node();
    seventh = new Node();
    eighth = new Node();
    ninth = new Node();
    tenth = new Node();
    head->data = 1;
    head->next = second;
    second->data = 2;
    second->next = third;
    third->data = 3;
    third->next = fourth;
    fourth->data = 4;
    fourth->next = fifth;
    fifth->data = 5;
    fifth->next = sixth;
    sixth->data = 6;
    sixth->next = seventh;
    seventh->data = 7;
    seventh->next = eighth;
    eighth->data = 8;
    eighth->next = ninth;
    ninth->data = 9;
    ninth->next = tenth;
    tenth->data = 10;
    tenth->next = NULL;

    printList(head);
    return 0;
}
```

Hasil percobaan:



3. **Percobaan 3 – Menambahkan Node Pada Singly Linked List**

Pada percobaan ini, akan dicoba untuk menambahkan node baru pada bagian awal SLL, pada bagian tengah SLL dan pada bagian akhir SLL menggunakan 2 jenis pemrograman, yaitu C dan C++.

a. **Menggunakan C**

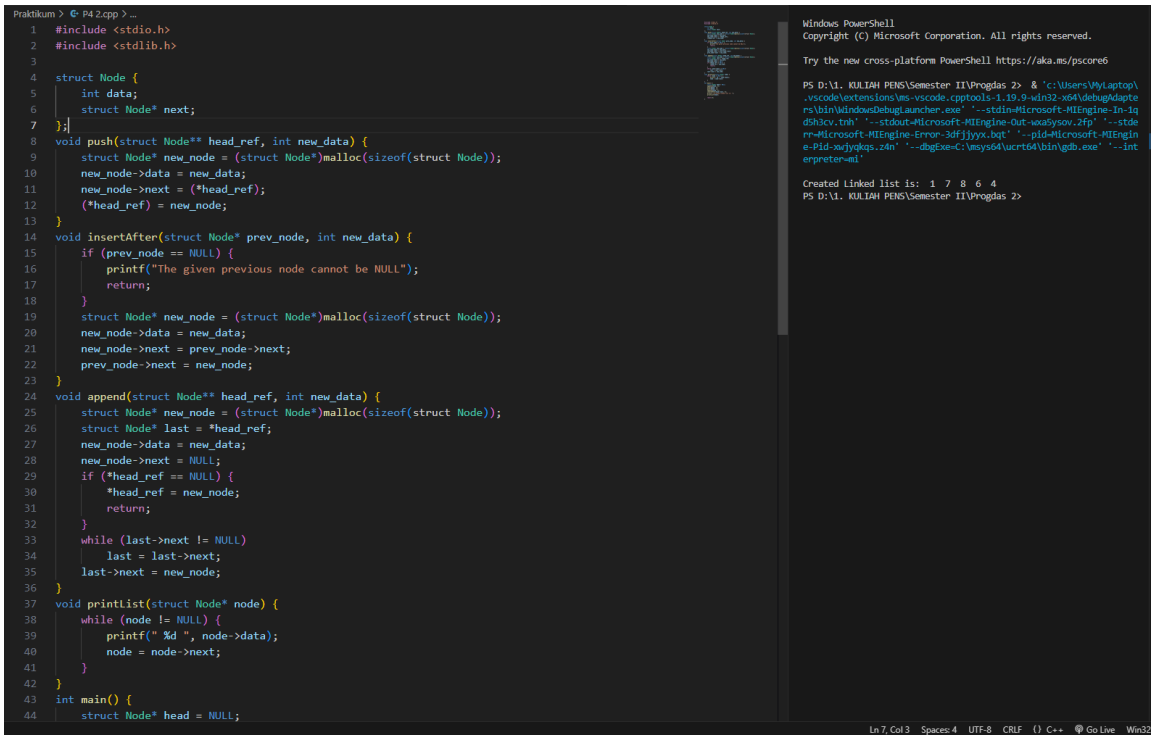
```
// A complete working C program to demonstrate all insertion methods
// on Linked List
#include <stdio.h>
#include <stdlib.h>

// A linked list node
struct Node
{
int data;
struct Node *next;
};

/* Given a reference (pointer to pointer) to the head of a list and
an int, inserts a new node on the front of the list. */
void push(struct Node** head_ref, int new_data)
{
	/* 1. allocate node */
	struct Node* new_node = (struct Node*) malloc(sizeof(struct Node));

	/* 2. put in the data */
	new_node->data = new_data;

	/* 3. Make next of new node as head */
	new_node->next = (*head_ref);

	/* 4. move the head to point to the new node */
	(*head_ref) = new_node;
}

/* Given a node prev_node, insert a new node after the given
prev_node */
void insertAfter(struct Node* prev_node, int new_data)
{
	/*1. check if the given prev_node is NULL */
	if (prev_node == NULL)
```

```
        {
        printf("the given previous node cannot be NULL");
        return;
        }

        /* 2. allocate new node */
        struct Node* new_node =(struct Node*) malloc(sizeof(struct Node));

        /* 3. put in the data */
        new_node->data = new_data;

        /* 4. Make next of new node as next of prev_node */
        new_node->next = prev_node->next;

        /* 5. move the next of prev_node as new_node */
        prev_node->next = new_node;
}

/* Given a reference (pointer to pointer) to the head
of a list and an int, appends a new node at the end */
void append(struct Node** head_ref, int new_data)
{
        /* 1. allocate node */
        struct Node* new_node = (struct Node*) malloc(sizeof(struct Node));

        struct Node *last = *head_ref; /* used in step 5*/

        /* 2. put in the data */
        new_node->data = new_data;

        /* 3. This new node is going to be the last node, so make next of
                it as NULL*/
        new_node->next = NULL;

        /* 4. If the Linked List is empty, then make the new node as head */
        if (*head_ref == NULL)
        {
        *head_ref = new_node;
        return;
        }
```

```
        /* 5. Else traverse till the last node */
        while (last->next != NULL)
                last = last->next;

        /* 6. Change the next of last node */
        last->next = new_node;
        return;
}

// This function prints contents of linked list starting from head
void printList(struct Node *node)
{
while (node != NULL)
{
        printf(" %d ", node->data);
        node = node->next;
}
}

/* Driver program to test above functions*/
int main()
{
/* Start with the empty list */
struct Node* head = NULL;

// Insert 6. So linked list becomes 6->NULL
append(&head, 6);

// Insert 7 at the beginning. So linked list becomes 7->6->NULL
push(&head, 7);

// Insert 1 at the beginning. So linked list becomes 1->7->6->NULL
push(&head, 1);

// Insert 4 at the end. So linked list becomes 1->7->6->4->NULL
append(&head, 4);

// Insert 8, after 7. So linked list becomes 1->7->8->6->4->NULL
insertAfter(head->next, 8);

printf("\n Created Linked list is: ");
```

```
printList(head);

return 0;
}
```

Hasil percobaan:



Dari percobaan di atas, coba tambahkan beberapa node baru misalnya 2 kali di awal SLL, 2 kali di tengah SLL dan 2 kali di akhir SLL secara acak! Amati hasilnya pada hasil cetak! Apakah hasilnya sudah sesuai dengan yang anda masukkan? Hasil cetak memasukkan juga alamat dari setiap node.

Kode percobaan:

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* next;
};
void push(struct Node** head_ref, int new_data) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct
Node));
    new_node->data = new_data;
    new_node->next = (*head_ref);
    (*head_ref) = new_node;
}
void insertAfter(struct Node* prev_node, int new_data) {
    if (prev_node == NULL) {
        printf("The given previous node cannot be NULL");
        return;
    }
```

```
    struct Node* new_node = (struct Node*)malloc(sizeof(struct
Node));
    new_node->data = new_data;
    new_node->next = prev_node->next;
    prev_node->next = new_node;
}
void append(struct Node** head_ref, int new_data) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct
Node));
    struct Node* last = *head_ref;
    new_node->data = new_data;
    new_node->next = NULL;
    if (*head_ref == NULL) {
        *head_ref = new_node;
        return;
    }
    while (last->next != NULL)
        last = last->next;
    last->next = new_node;
}
void printList(struct Node* node) {
    while (node != NULL) {
        printf(" %d (%p) ", node->data, node);
        node = node->next;
    }
}
int main() {
    struct Node* head = NULL;
    push(&head, 10);
    push(&head, 20);
    struct Node* middle = head->next;
    insertAfter(middle, 30);
    insertAfter(middle, 40);
    append(&head, 50);
    append(&head, 60);
    printf("Created Linked list is:");
    printList(head);

    return 0;
}
```

Hasil percobaan:

Praktikum > P4 2.1.cpp > main()

```cpp
void insertAfter(struct Node* prev_node, int new_data) {
        return;
    }
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    new_node->data = new_data;
    new_node->next = prev_node->next;
    prev_node->next = new_node;
}
void append(struct Node** head_ref, int new_data) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    struct Node* last = *head_ref;
    new_node->data = new_data;
    new_node->next = NULL;
    if (*head_ref == NULL) {
        *head_ref = new_node;
        return;
    }
    while (last->next != NULL)
        last = last->next;
    last->next = new_node;
}
void printList(struct Node* node) {
    while (node != NULL) {
        printf(" %d (%p) ", node->data, node);
        node = node->next;
    }
}
int main() {
    struct Node* head = NULL;
    push(&head, 10);
    push(&head, 20);
    struct Node* middle = head->next;
    insertAfter(middle, 30);
    insertAfter(middle, 40);
    append(&head, 50);
    append(&head, 60);
    printf("Created Linked list is:");
    printList(head);

    return 0;
}
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS D:\1. KULIAH PENS\Semester II\Progdas 2>  & 'c:\Users\MyLaptop\.vscode\extensions\ms-vscode.cpptools-1.19.9-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-ycivalna.43d' '--stdout=Microsoft-MIEngine-Out-hsk4pbyy.5yg' '--stderr=Microsoft-MIEngine-Error-clllvyn1.ddy' '--pid=Microsoft-MIEngine-Pid-4w2i2yqj.vbx' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=mi'
Created Linked list is: 20 (0000000000E5F50)  10 (0000000000E3DF0)  40 (0000000000E61B0)  30 (0000000000E6170)  50 (0000000000E61F0)  60 (0000000000E6230)
PS D:\1. KULIAH PENS\Semester II\Progdas 2>
```

b. Menggunakan C++

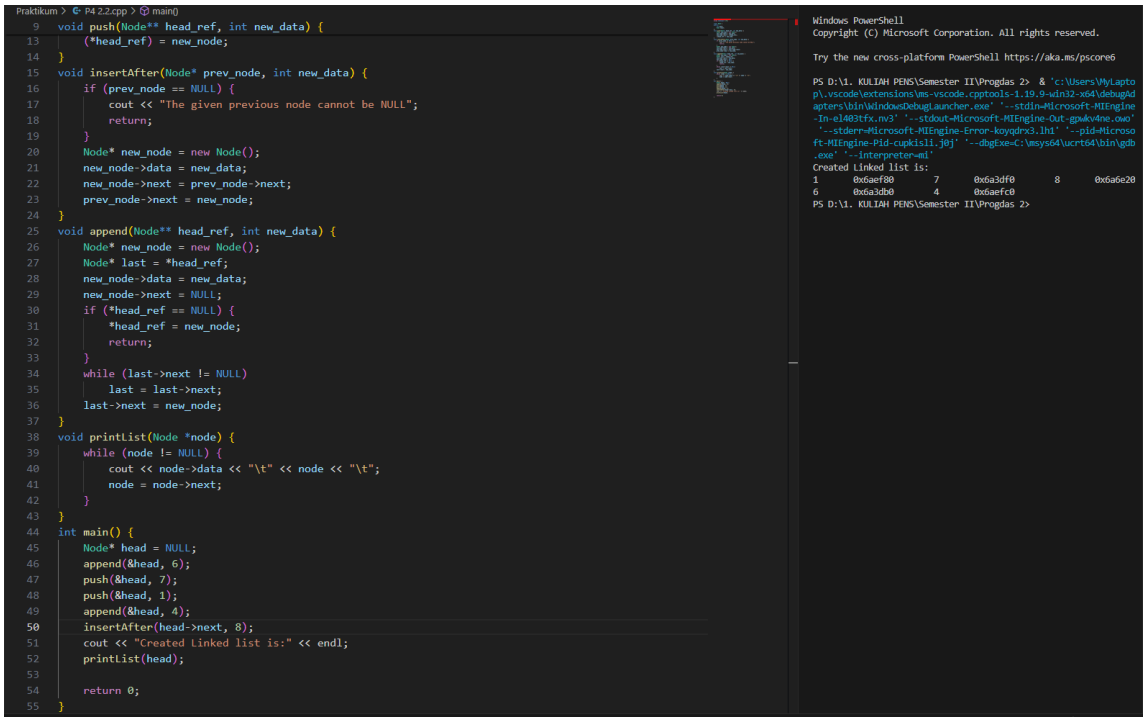
```
// A complete working C++ program to demonstrate
// all insertion methods on Linked List
#include <bits/stdc++.h>
using namespace std;

// A linked list node
class Node
{
	public:
	int data;
	Node *next;
};

/* Given a reference (pointer to pointer)
to the head of a list and an int, inserts
a new node on the front of the list. */
void push(Node** head_ref, int new_data)
{
	/* 1. allocate node */
	Node* new_node = new Node();

	/* 2. put in the data */
	new_node->data = new_data;

	/* 3. Make next of new node as head */
	new_node->next = (*head_ref);

	/* 4. move the head to point to the new node */
	(*head_ref) = new_node;
}
```

```
/* Given a node prev_node, insert a new node after the given
prev_node */
void insertAfter(Node* prev_node, int new_data)
{
	/*1. check if the given prev_node is NULL */
	if (prev_node == NULL)
	{
		cout<<"the given previous node cannot be NULL";
		return;
	}

	/* 2. allocate new node */
	Node* new_node = new Node();

	/* 3. put in the data */
	new_node->data = new_data;

	/* 4. Make next of new node as next of prev_node */
	new_node->next = prev_node->next;

	/* 5. move the next of prev_node as new_node */
	prev_node->next = new_node;
}

/* Given a reference (pointer to pointer) to the head
of a list and an int, appends a new node at the end */
void append(Node** head_ref, int new_data)
{
	/* 1. allocate node */
	Node* new_node = new Node();

	Node *last = *head_ref; /* used in step 5*/

	/* 2. put in the data */
	new_node->data = new_data;

	/* 3. This new node is going to be
	the last node, so make next of
	it as NULL*/
	new_node->next = NULL;
```

```
	/* 4. If the Linked List is empty,
	then make the new node as head */
	if (*head_ref == NULL)
	{
		*head_ref = new_node;
		return;
	}

	/* 5. Else traverse till the last node */
	while (last->next != NULL)
		last = last->next;

	/* 6. Change the next of last node */
	last->next = new_node;
	return;
}

// This function prints contents of
// linked list starting from head
void printList(Node *node)
{
	while (node != NULL)
	{
		cout<<" "<<node->data;
		node = node->next;
	}
}

/* Driver code*/
int main()
{
	/* Start with the empty list */
	Node* head = NULL;

	// Insert 6. So linked list becomes 6->NULL
	append(&head, 6);

	// Insert 7 at the beginning.
	// So linked list becomes 7->6->NULL
	push(&head, 7);
```

```
        // Insert 1 at the beginning.
        // So linked list becomes 1->7->6->NULL
        push(&head, 1);

        // Insert 4 at the end. So
        // linked list becomes 1->7->6->4->NULL
        append(&head, 4);

        // Insert 8, after 7. So linked
        // list becomes 1->7->8->6->4->NULL
        insertAfter(head->next, 8);

        cout<<"Created Linked list is: ";
        printList(head);

        return 0;
}
```

Hasil percobaan:



Dari percobaan di atas, coba tambahkan beberapa node baru misalnya 2 kali di awal SLL, 2 kali di tengah SLL dan 2 kali di akhir SLL secara acak! Amati hasilnya pada hasil cetak! Apakah hasilnya sudah sesuai dengan yang anda masukkan? Hasil cetak memasukkan juga alamat dari setiap node.

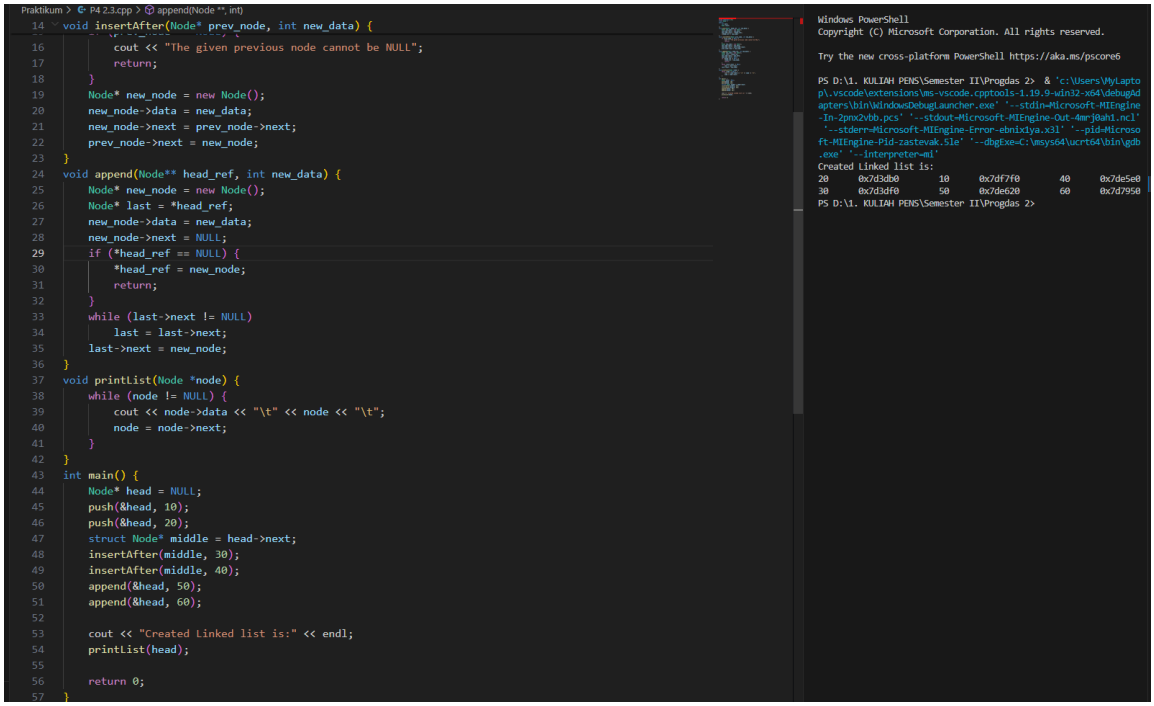
Kode percobaan:

```
#include <bits/stdc++.h>
using namespace std;
class Node {
```

```
public:
    int data;
    Node *next;
};
void push(Node** head_ref, int new_data) {
    Node* new_node = new Node();
    new_node->data = new_data;
    new_node->next = (*head_ref);
    (*head_ref) = new_node;
}
void insertAfter(Node* prev_node, int new_data) {
    if (prev_node == NULL) {
        cout << "The given previous node cannot be NULL";
        return;
    }
    Node* new_node = new Node();
    new_node->data = new_data;
    new_node->next = prev_node->next;
    prev_node->next = new_node;
}
void append(Node** head_ref, int new_data) {
    Node* new_node = new Node();
    Node* last = *head_ref;
    new_node->data = new_data;
    new_node->next = NULL;
    if (*head_ref == NULL) {
        *head_ref = new_node;
        return;
    }
    while (last->next != NULL)
        last = last->next;
    last->next = new_node;
}
void printList(Node *node) {
    while (node != NULL) {
        cout << node->data << "\t" << node << "\t";
        node = node->next;
    }
}
int main() {
    Node* head = NULL;
    push(&head, 10);
    push(&head, 20);
    struct Node* middle = head->next;
    insertAfter(middle, 30);
```

```
    insertAfter(middle, 40);
    append(&head, 50);
    append(&head, 60);

    cout << "Created Linked list is:" << endl;
    printList(head);

    return 0;
}
```

Hasil percobaan:



4. Percobaan 4 – Menghapus Node Pada Singly Linked List

Pada percobaan ini, node tertentu dari sebuah SLL dapat dihapus berdasarkan 2 cara, yaitu dihapus menggunakan nilai datanya dan dihapus menggunakan nomor node.

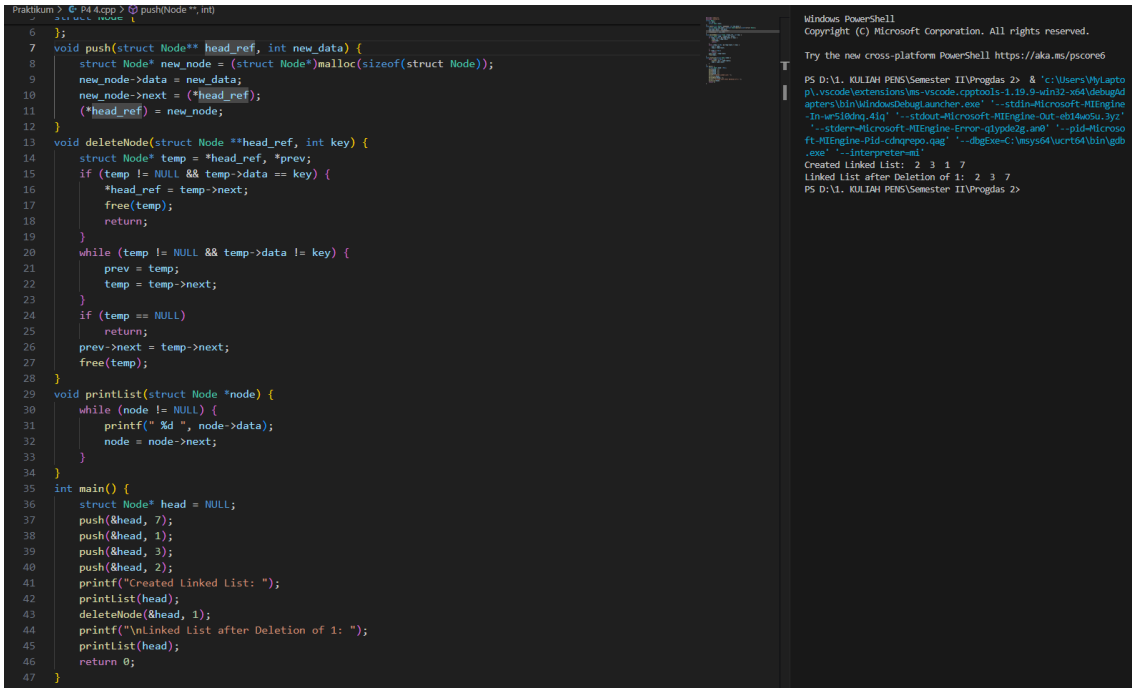
a. Penghapusan menggunakan nilai data

```
// A complete working C program to demonstrate deletion in singly
// linked list
#include <stdio.h>
#include <stdlib.h>

// A linked list node
struct Node
{
        int data;
        struct Node *next;
};
```

```
/* Given a reference (pointer to pointer) to the head of a list
and an int, inserts a new node on the front of the list. */
void push(struct Node** head_ref, int new_data)
{
	struct Node* new_node = (struct Node*) malloc(sizeof(struct Node));
	new_node->data = new_data;
	new_node->next = (*head_ref);
	(*head_ref) = new_node;
}

/* Given a reference (pointer to pointer) to the head of a list
and a key, deletes the first occurrence of key in linked list */
void deleteNode(struct Node **head_ref, int key)
{
	// Store head node
	struct Node* temp = *head_ref, *prev;

	// If head node itself holds the key to be deleted
	if (temp != NULL && temp->data == key)
	{
		*head_ref = temp->next; // Changed head
		free(temp);			// free old head
		return;
	}

	// Search for the key to be deleted, keep track of the
	// previous node as we need to change 'prev->next'
	while (temp != NULL && temp->data != key)
	{
		prev = temp;
		temp = temp->next;
	}

	// If key was not present in linked list
	if (temp == NULL) return;

	// Unlink the node from linked list
	prev->next = temp->next;

	free(temp); // Free memory
}
```

```
// This function prints contents of linked list starting from
// the given node
void printList(struct Node *node)
{
	while (node != NULL)
	{
		printf(" %d ", node->data);
		node = node->next;
	}
}

/* Drier program to test above functions*/
int main()
{
	/* Start with the empty list */
	struct Node* head = NULL;

	push(&head, 7);
	push(&head, 1);
	push(&head, 3);
	push(&head, 2);

	puts("Created Linked List: ");
	printList(head);
	deleteNode(&head, 1);
	puts("\nLinked List after Deletion of 1: ");
	printList(head);
	return 0;
}
```

Hasil percobaan:



Dari percobaan di atas, cobalah untuk menambahkan 5 node baru terlebih dahulu (lokasi boleh acak, bisa di depan, di tengah ataupun di belakang), kemudian lakukan penghapusan terhadap 3 data yang baru anda masukkan secara acak!

Kode percobaan:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};
void push(struct Node** head_ref, int new_data) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct
Node));
    new_node->data = new_data;
    new_node->next = (*head_ref);
    (*head_ref) = new_node;
}
void deleteNode(struct Node **head_ref, int key) {
    struct Node* temp = *head_ref, *prev;
    if (temp != NULL && temp->data == key) {
        *head_ref = temp->next;
        free(temp);
        return;
    }
    while (temp != NULL && temp->data != key) {
        prev = temp;
        temp = temp->next;
    }
```

```c
    if (temp == NULL)
        return;

    prev->next = temp->next;
    free(temp);
}
void printList(struct Node *node) {
    while (node != NULL) {
        printf(" %d ", node->data);
        node = node->next;
    }
}
int main() {
    struct Node* head = NULL;
    push(&head, 7);
    push(&head, 5);
    push(&head, 9);
    push(&head, 2);
    push(&head, 4);
    printf("Linked List setelah penambahan 5 node baru: ");
    printList(head);
    deleteNode(&head, 5);
    deleteNode(&head, 9);
    deleteNode(&head, 4);
    printf("\nLinked List setelah penghapusan: ");
    printList(head);

    return 0;
}
```

Hasil percobaan:

Praktikum > P4 4.1.cpp > deleteNode(Node **, int)

```cpp
void push(struct Node** head_ref, int new_data) {
    new_node->next = (*head_ref);
    (*head_ref) = new_node;
}
void deleteNode(struct Node **head_ref, int key) {
    struct Node* temp = *head_ref, *prev;
    if (temp != NULL && temp->data == key) {
        *head_ref = temp->next;
        free(temp);
        return;
    }
    while (temp != NULL && temp->data != key) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL)
        return;

    prev->next = temp->next;
    free(temp);
}
void printList(struct Node *node) {
    while (node != NULL) {
        printf(" %d ", node->data);
        node = node->next;
    }
}
int main() {
    struct Node* head = NULL;
    push(&head, 7);
    push(&head, 5);
    push(&head, 9);
    push(&head, 2);
    push(&head, 4);
    printf("Linked List setelah penambahan 5 node baru: ");
    printList(head);
    deleteNode(&head, 5);
    deleteNode(&head, 9);
    deleteNode(&head, 4);
    printf("\nLinked List setelah penghapusan: ");
    printList(head);

    return 0;
}
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS D:\1. KULIAH PENS\Semester II\Progdas 2>  & 'c:\Users\MyLapto
p\.vscode\extensions\ms-vscode.cpptools-1.19.9-win32-x64\debugAd
apters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine
-In-x4q15sr2.o1b' '--stdout=Microsoft-MIEngine-Out-vmwg3srk.fgl'
 '--stderr=Microsoft-MIEngine-Error-evwi3jh2.qyd' '--pid=Microso
ft-MIEngine-Pid-30gkp3zc.zg3' '--dbgExe=C:\msys64\ucrt64\bin\gdb
.exe' '--interpreter=mi'
Linked List setelah penambahan 5 node baru:  4  2  9  5  7
Linked List setelah penghapusan:  2  7
PS D:\1. KULIAH PENS\Semester II\Progdas 2>
```

b. Penghapusan menggunakan nomor node

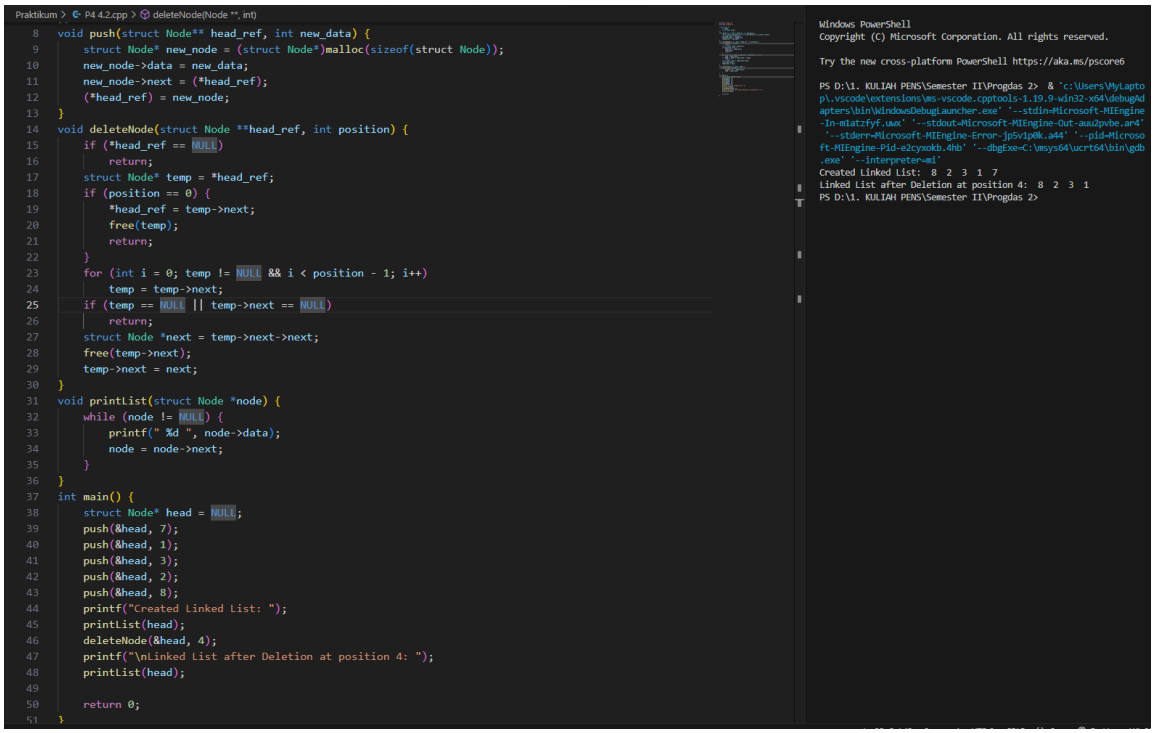
```cpp
// A complete working C program to delete a node in a linked list
// at a given position
#include <stdio.h>
#include <stdlib.h>
```

```
// A linked list node
struct Node
{
	int data;
	struct Node *next;
};

/* Given a reference (pointer to pointer) to the head of a list
and an int, inserts a new node on the front of the list. */
void push(struct Node** head_ref, int new_data)
{
	struct Node* new_node = (struct Node*) malloc(sizeof(struct Node));
	new_node->data = new_data;
	new_node->next = (*head_ref);
	(*head_ref) = new_node;
}

/* Given a reference (pointer to pointer) to the head of a list
and a position, deletes the node at the given position */
void deleteNode(struct Node **head_ref, int position)
{
// If linked list is empty
if (*head_ref == NULL)
	return;

// Store head node
struct Node* temp = *head_ref;

	// If head needs to be removed
	if (position == 0)
	{
		*head_ref = temp->next; // Change head
		free(temp);		// free old head
		return;
	}

	// Find previous node of the node to be deleted
	for (int i=0; temp!=NULL && i<position-1; i++)
		temp = temp->next;

	// If position is more than number of ndoes
```

```
	if (temp == NULL || temp->next == NULL)
		return;

	// Node temp->next is the node to be deleted
	// Store pointer to the next of node to be deleted
	struct Node *next = temp->next->next;

	// Unlink the node from linked list
	free(temp->next); // Free memory

	temp->next = next; // Unlink the deleted node from list
}

// This function prints contents of linked list starting from
// the given node
void printList(struct Node *node)
{
	while (node != NULL)
	{
		printf(" %d ", node->data);
		node = node->next;
	}
}

/* Drier program to test above functions*/
int main()
{
	/* Start with the empty list */
	struct Node* head = NULL;

	push(&head, 7);
	push(&head, 1);
	push(&head, 3);
	push(&head, 2);
	push(&head, 8);

	puts("Created Linked List: ");
	printList(head);
	deleteNode(&head, 4);
	puts("\nLinked List after Deletion at position 4: ");
	printList(head);
```

```
        return 0;
}
```

Hasil percobaan:



Dari percobaan di atas, cobalah untuk menambahkan 5 node baru terlebih dahulu (lokasi boleh acak, bisa di depan, di tengah ataupun di belakang), kemudian lakukan penghapusan terhadap 3 data yang baru anda masukkan secara acak!

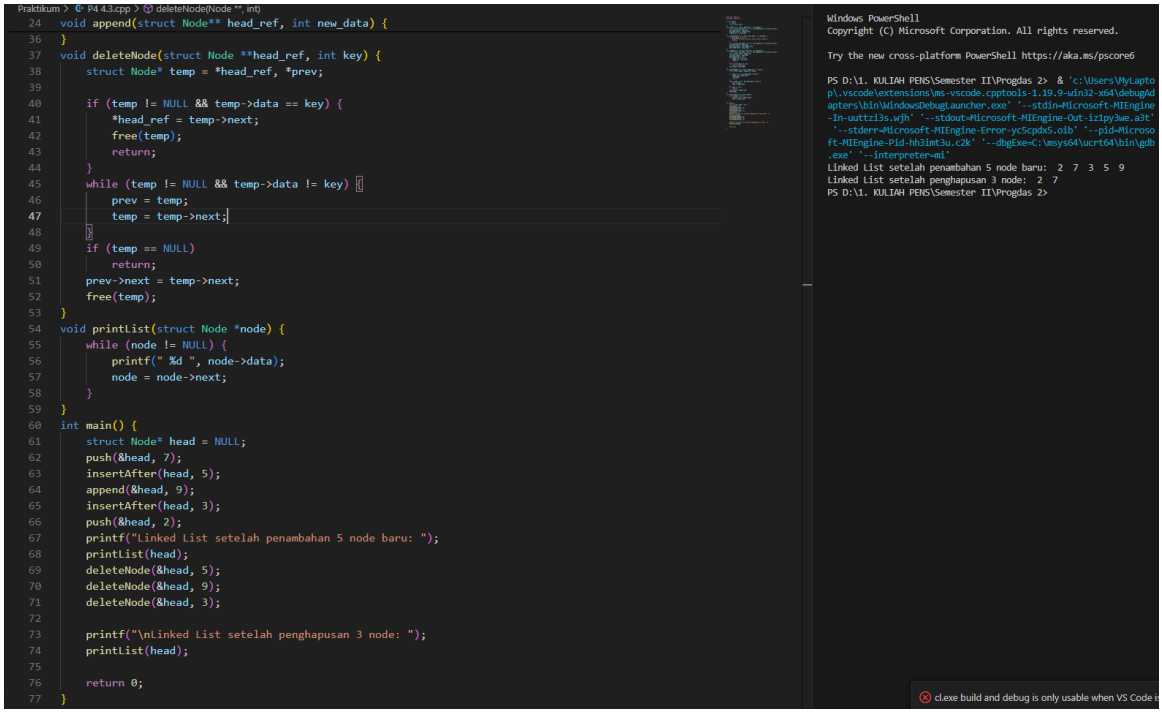
Kode pemograman:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};
void push(struct Node** head_ref, int new_data) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct
Node));
    new_node->data = new_data;
    new_node->next = (*head_ref);
    (*head_ref) = new_node;
}
void insertAfter(struct Node* prev_node, int new_data) {
    if (prev_node == NULL) {
        printf("The given previous node cannot be NULL");
        return;
    }
```

```
    struct Node* new_node = (struct Node*)malloc(sizeof(struct
Node));
    new_node->data = new_data;
    new_node->next = prev_node->next;
    prev_node->next = new_node;
}
void append(struct Node** head_ref, int new_data) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct
Node));
    struct Node* last = *head_ref;
    new_node->data = new_data;
    new_node->next = NULL;
    if (*head_ref == NULL) {
        *head_ref = new_node;
        return;
    }
    while (last->next != NULL)
        last = last->next;
    last->next = new_node;
}
void deleteNode(struct Node **head_ref, int key) {
    struct Node* temp = *head_ref, *prev;

    if (temp != NULL && temp->data == key) {
        *head_ref = temp->next;
        free(temp);
        return;
    }
    while (temp != NULL && temp->data != key) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL)
        return;
    prev->next = temp->next;
    free(temp);
}
void printList(struct Node *node) {
    while (node != NULL) {
        printf(" %d ", node->data);
        node = node->next;
    }
}
int main() {
    struct Node* head = NULL;
```

```c
    push(&head, 7);
    insertAfter(head, 5);
    append(&head, 9);
    insertAfter(head, 3);
    push(&head, 2);
    printf("Linked List setelah penambahan 5 node baru: ");
    printList(head);
    deleteNode(&head, 5);
    deleteNode(&head, 9);
    deleteNode(&head, 3);

    printf("\nLinked List setelah penghapusan 3 node: ");
    printList(head);

    return 0;
}
```

Hasil percobaan:



c. **Tugas:** Dari kedua model penghapusan node di atas, manakah yang lebih cepat? Menggunakan key/nilai data atau menggunakan nomor node? Tunjukkan dalam waktu (millisecond)!

Anda boleh menggunakan library chrono seperti dibawah ini:

```
#include <iostream>
#include <chrono>

using namespace std;
int main()
{
   chrono::steady_clock sc;   // create an object of `steady_clock` class
   auto start = sc.now();     // start timer

   // do stuff....

   auto end = sc.now();       // end timer (starting & ending is done by measuring the time
   auto time_span = static_cast<chrono::duration<double>>(end - start);   // measure time s
   cout<<"Operation took: "<<time_span.count()<<" seconds !!!";
   return 0;
}
```

Atau menggunakan library ctime seperti di bawah ini:

```
#include <iostream>
#include <cstdio>
#include <ctime>

int main()
{
    std::clock_t start;
    double duration;

    start = std::clock(); // get current time

    // Do your stuff here

    duration = ( std::clock() - start ) / (double) CLOCKS_PER_SEC;

    std::cout << "Operation took "<< duration << "seconds" << std::endl;

    return 0;
}
```

Di dalam “do stuff” adalah fungsi deleteNode(&head,X). Artinya, variable start–duration atau start–end mengapit fungsi deleteNode(&head,X) saja.

```
#include <iostream>
# #include <iostream>
#include <chrono>
#include <ctime>
using namespace std;

struct Node {
    int data;
    Node *next;
};
void push(Node** head_ref, int new_data) {
    Node* new_node = new Node();
    new_node->data = new_data;
    new_node->next = (*head_ref);
    (*head_ref) = new_node;
```

```
}
void deleteNodeByKey(Node **head_ref, int key) {
    Node* temp = *head_ref, *prev = nullptr;
    if (temp != nullptr && temp->data == key) {
        *head_ref = temp->next;
        delete temp;
        return;
    }
    while (temp != nullptr && temp->data != key) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == nullptr)
        return;
    prev->next = temp->next;
    delete temp;
}
void deleteNodeByPosition(Node **head_ref, int position) {
    if (*head_ref == nullptr)
        return;
    Node* temp = *head_ref;
    if (position == 0) {
        *head_ref = temp->next;
        delete temp;
        return;
    }
    for (int i = 0; temp != nullptr && i < position - 1; i++)
        temp = temp->next;
    if (temp == nullptr || temp->next == nullptr)
        return;
    Node* next = temp->next->next;
    delete temp->next;
    temp->next = next;
}
void printList(Node *node) {
    while (node != nullptr) {
        cout << node->data << " ";
        node = node->next;
    }
}
int main() {
    Node* head = nullptr;
    for (int i = 0; i < 10000; i++)
        push(&head, i);
    auto startKey = chrono::steady_clock::now();
```

```
    deleteNodeByKey(&head, 5000);
    auto endKey = chrono::steady_clock::now();
    auto startPosition = chrono::steady_clock::now();
    deleteNodeByPosition(&head, 5000);
    auto endPosition = chrono::steady_clock::now();
    chrono::duration<double, milli> elapsedKey = endKey -
startKey;
    chrono::duration<double, milli> elapsedPosition =
endPosition - startPosition;
    cout << "Time eksekusi model penghapusan berdasarkan key: "
<< elapsedKey.count() << " milliseconds\n";
    cout << "Waktu eksekusi model penghapusan berdasarkan nomor
node: " << elapsedPosition.count() << " milliseconds\n";
    return 0;
}
```

Dari hasil tersebut, dapat disimpulkan bahwa penghapusan berdasarkan key (nilai data) lebih cepat daripada penghapusan berdasarkan nomor node dalam kasus yang di uji. Hal ini mungkin disebabkan oleh algoritma pencarian yang berbeda digunakan dalam setiap metode penghapusan. Dalam penghapusan berdasarkan key, algoritma hanya perlu mencari node dengan nilai data yang sesuai, sedangkan dalam penghapusan berdasarkan nomor node, algoritma harus melintasi linked list hingga mencapai posisi node yang akan dihapus.

Hasil percobaan:

```
PS D:\1. KULIAH PENS\Semester II\Progdas 2>  & 'c:\Users\MyLaptop\.vscode\extension
s\ms-vscode.cpptools-1.19.9-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '
--stdin=Microsoft-MIEngine-In-r5vtdmkz.mft' '--stdout=Microsoft-MIEngine-Out-02g4ok
bp.5hc' '--stderr=Microsoft-MIEngine-Error-wvs1p0jh.og3' '--pid=Microsoft-MIEngine-
Pid-53hfhrt3.f2k' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=mi'
Waktu yang diperlukan untuk menghapus berdasarkan key: 0.0121 milliseconds
Waktu yang diperlukan untuk menghapus berdasarkan nomor node: 0.1328 milliseconds
PS D:\1. KULIAH PENS\Semester II\Progdas 2>
```